VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

NAME: KAVINILA C

REG. NO: 23MIS0004

COURSE NAME: AGILE DEVELOPMENT PROCESS AND DEVOPS LAB.

LAB ASSESSMENT-2

COURSE CODE: ISWE406P

SLOT: L49+L50

FACULTY: SWETA BHATTACHARYA

JENKINS

1. Learning Goal

Understand what Jenkins is and run a job successfully.

Stepwise Tasks

1.1.What is CI/CD? (Real-world example)

CI – Continuous Integration

- **Developers push code many times a day**
- **System automatically:**
  - **pulls code**
  - **builds it**
  - **tests it**

CD – Continuous Delivery / Deployment

- **After CI succeeds:**
  - **application is packaged**
  - **deployed to server automatically**

Real-world analogy

- **Google Docs:**
  - **typing → auto-save (CI)**
  - **document updates instantly for others (CD)**

Where Jenkins fits
Jenkins is the automation engine that performs these steps without human involvement.

1.2.Install Jenkins (Windows / Linux)

Step 1: Install Java (mandatory)

Jenkins will not run without Java.

1. **Download Java 17 (LTS) – Temurin / Adoptium**
2. **Install**
3. **Verify:**

java -version

Step 2: Install Jenkins

1. **Download Jenkins Windows (.msi)**
2. **Run installer**
3. **Keep defaults:**
   - **Port: 8080**
   - **Run as Windows service ✓**

Step 3: Start Jenkins

4. **Open browser:`http://localhost:8080`**

Step 4: Unlock Jenkins

1. **Jenkins asks for password**
2. **Go to:**

C:\Program Files\Jenkins\secrets\initialAdminPassword

3. **Copy → paste → Continue**

Step 5: Plugin installation

Choose:
✓ Install suggested plugins

Create:

- **Username**
- **Password**

Jenkins dashboard opens.

1.3.Jenkins UI overview

Dashboard – lists all jobs

New Item – create job

Manage Jenkins – configuration, plugins, users

Build History – previous runs

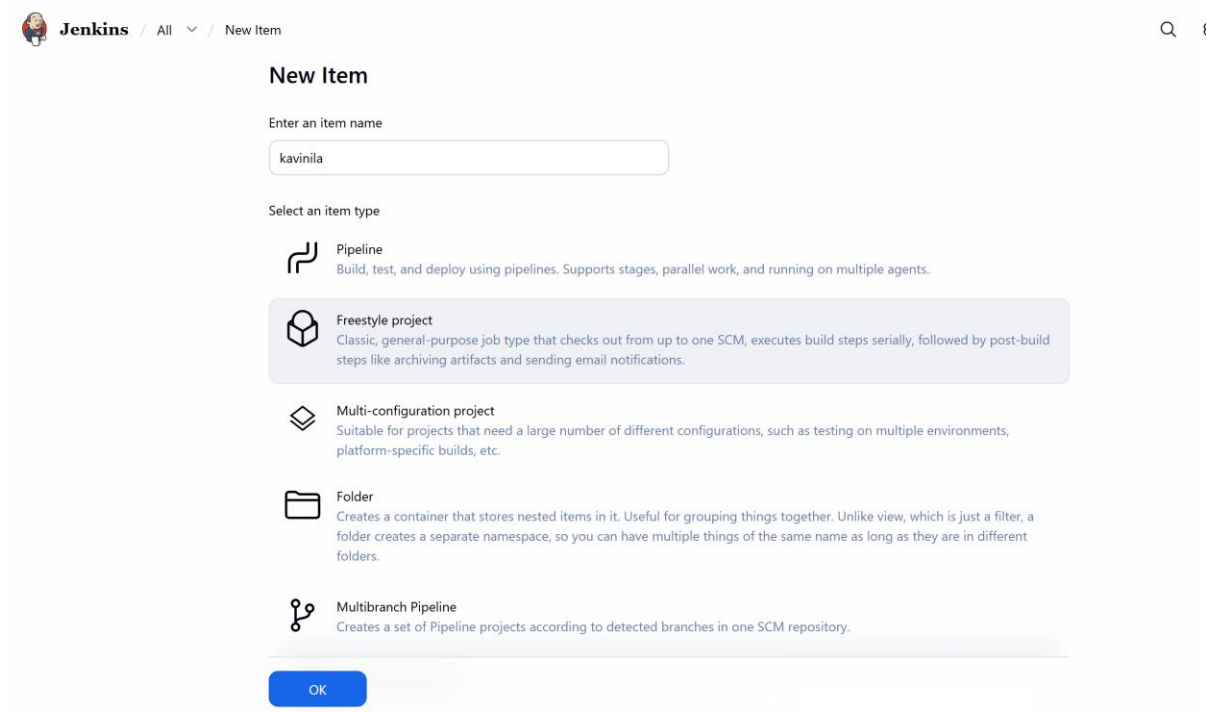
1.4.Create Freestyle Job

Click New Item

Name: `Hello-Jenkins`

Select: Freestyle project

Click OK



1.5.Add simple build step:

Windows: `echo Hello Jenkins`

1. **Run job manually**
2. **View:**
   - **Console Output**

Jenkins / kavinila / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Add timestamps to the Console Output

Inspect build log for published build scans

Terminate a build if it's stuck

With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command ?

Command

See the list of available environment variables

```
echo Hello Jenkins
```

Advanced

+ Add build step

Save Apply

Jenkins / kavinila / #1

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Open Blue Ocean

Console

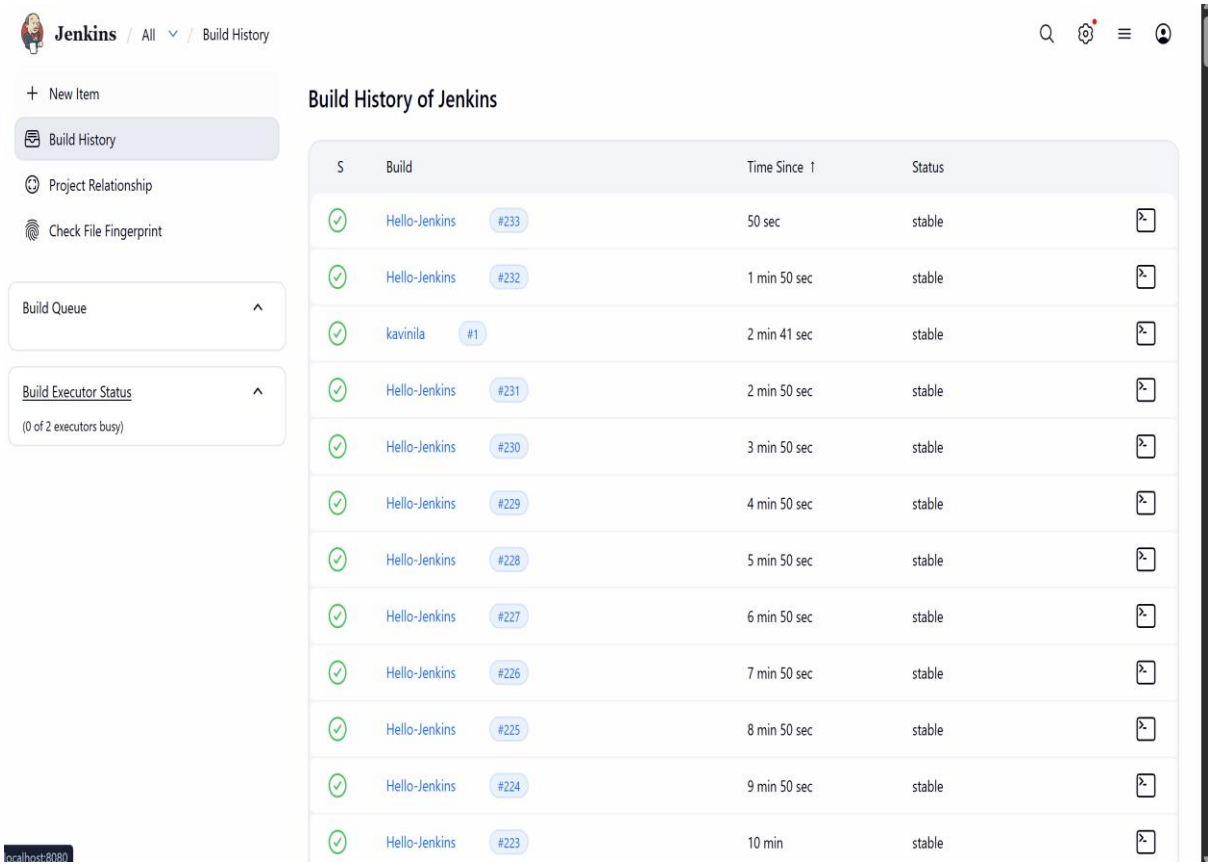
Download Copy View as plain text

```
Started by user Kavinila
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\kavinila
[kavinila] $ cmd /c call C:\WINDOWS\TEMP\jenkins16981979745782192001.bat

C:\ProgramData\Jenkins\.jenkins\workspace\kavinila>echo Hello Jenkins
Hello Jenkins

C:\ProgramData\Jenkins\.jenkins\workspace\kavinila>exit 0
Finished: SUCCESS
```

1.6.Build History



Outcome

A learner can install Jenkins and run a basic job.

2. Learning Goal

Work with source code and build automation

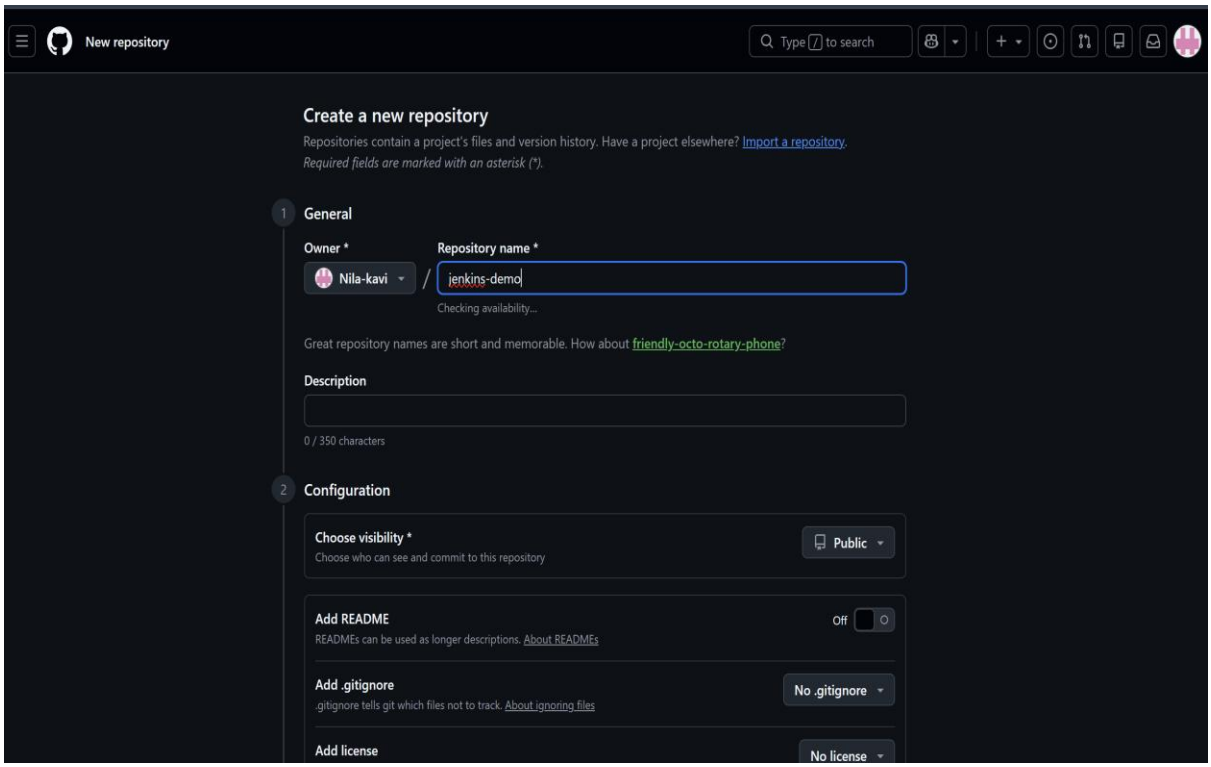
Stepwise Tasks

2.1.Install Git

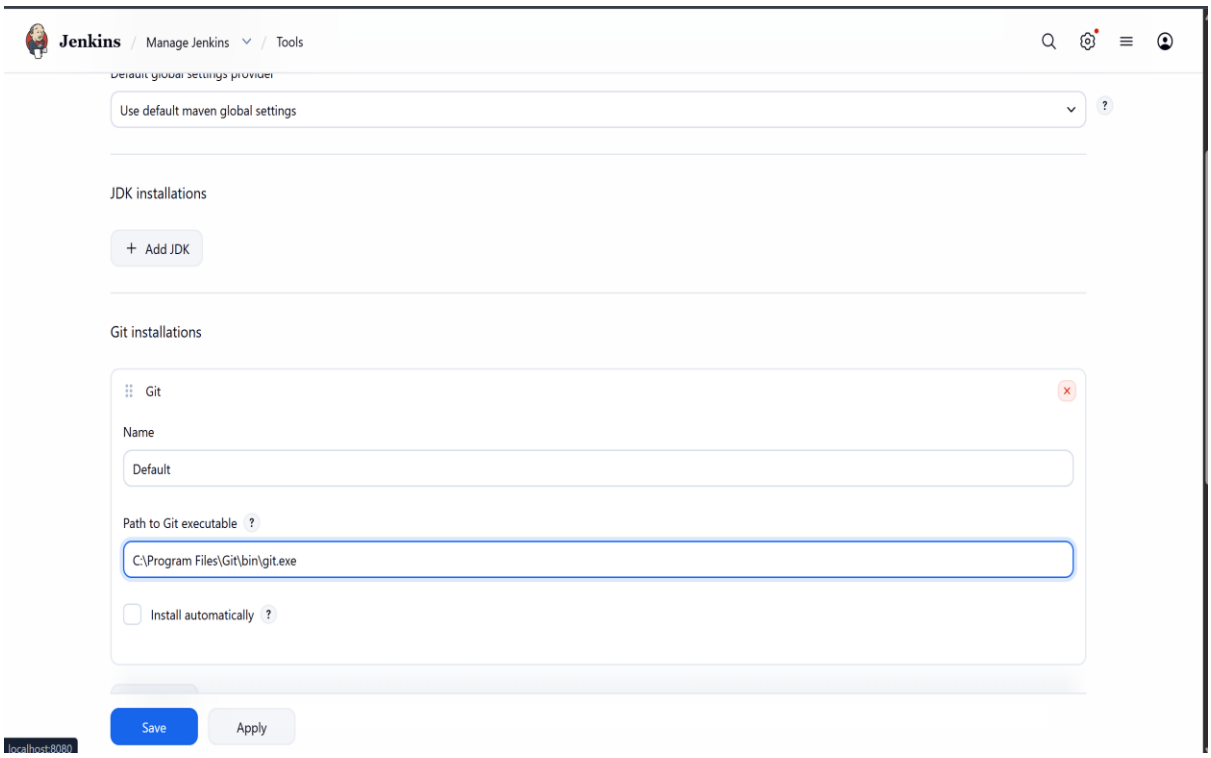
Install Git for Windows
Verify:

git --version

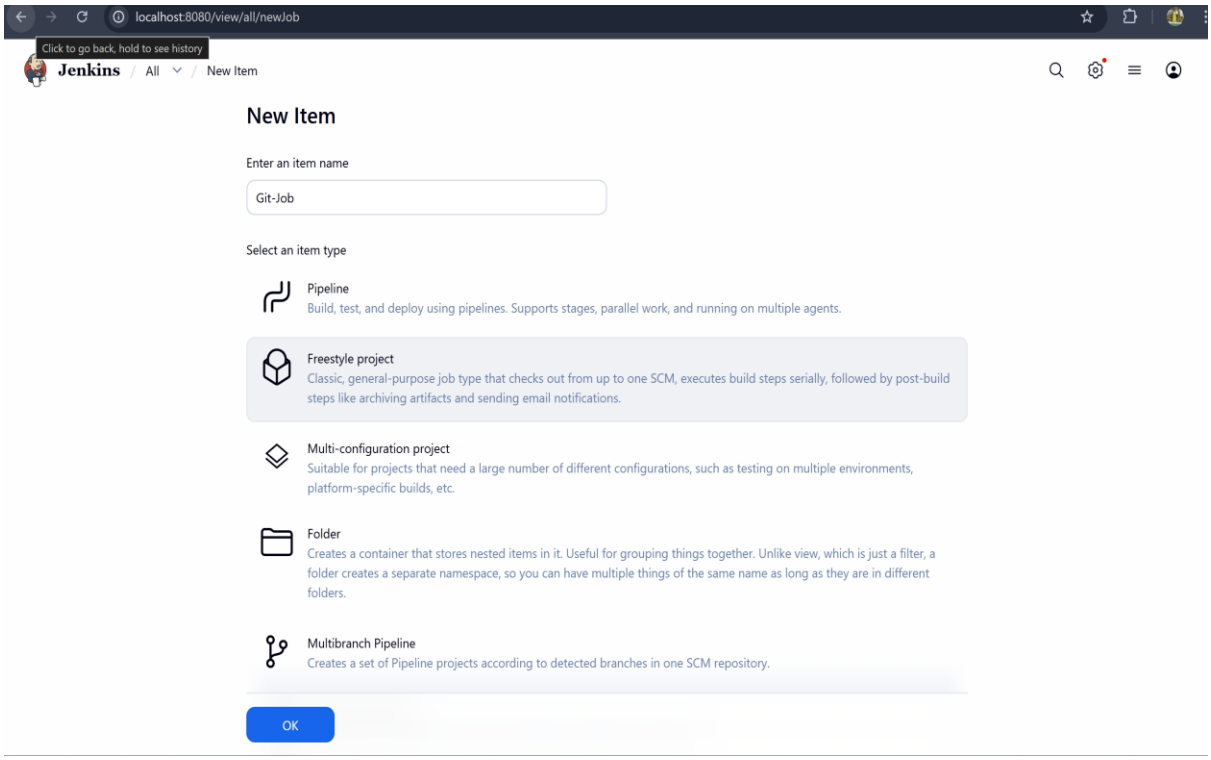
2.2. Create GitHub repository



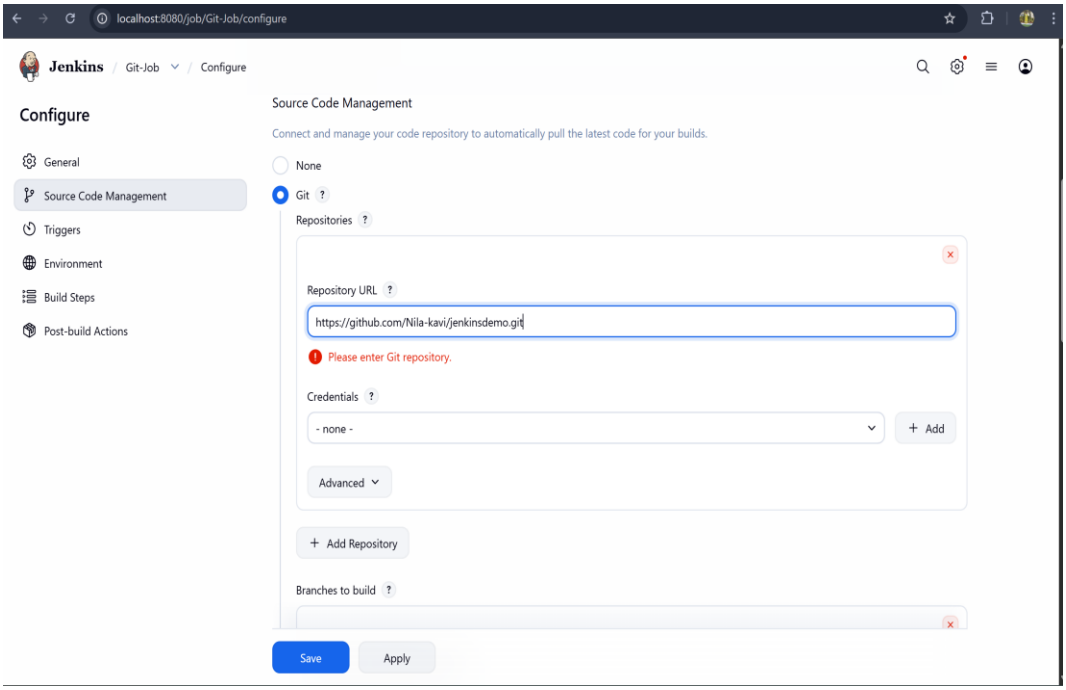
2.3. Configure Git in Jenkins



2.4. Create Freestyle Job:



Source Code Management → Git



2.5. Add build step:

- **Windows batch / Shell**

Jenkins / Git-Job / Configure

Configure

General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions

Use secret text(s) or file(s) ?
Add timestamps to the Console Output
Inspect build log for published build scans
Terminate a build if it's stuck
With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command ?

Command

See the list of available environment variables

```
echo Code pulled successfully
dir
```

Advanced

+ Add build step

Save Apply

2.5.Poll SCM / Webhook trigger

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- [ ] Trigger builds remotely (e.g., from scripts) ?
- [ ] Build after other projects are built ?
- [ ] Build periodically ?
- [ ] GitHub hook trigger for GITScm polling ?
- [x] Poll SCM ?

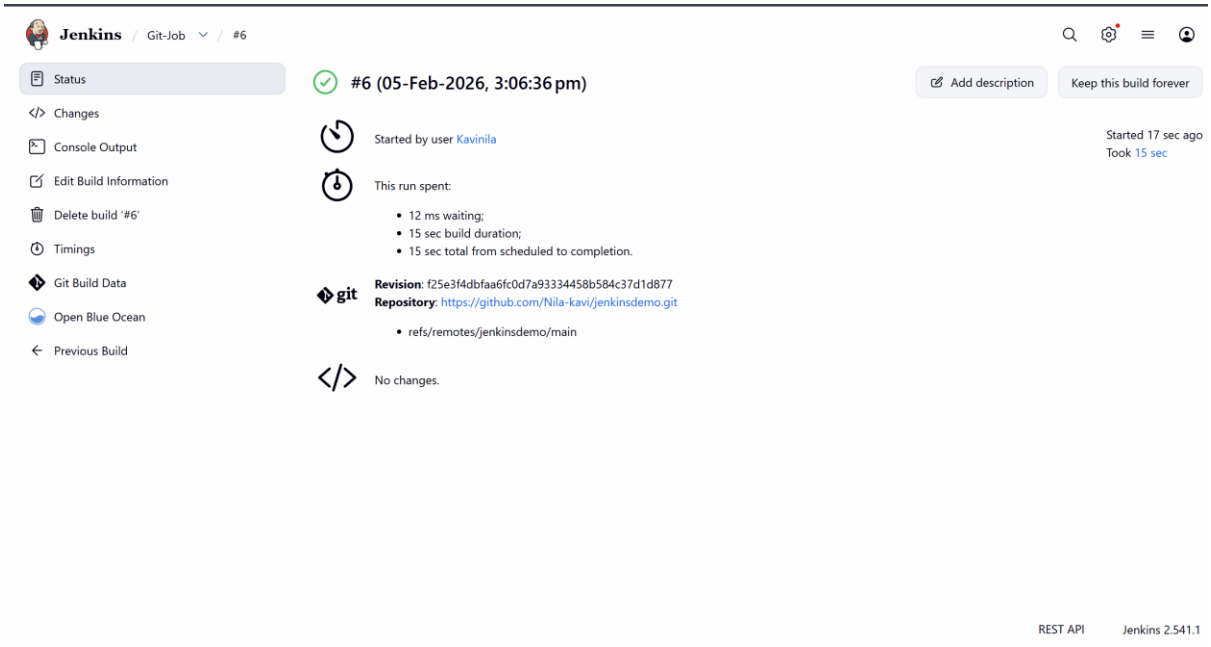
Schedule ?

H/2 * * * *

Would last have run at Thursday, 5 February, 2026, 4:02:00 pm India Standard Time; would next run at Thursday, 5 February, 2026, 4:04:00 pm India Standard Time.

- [ ] Ignore post-commit hooks ?

2.6.Build automatically on code change



Outcome

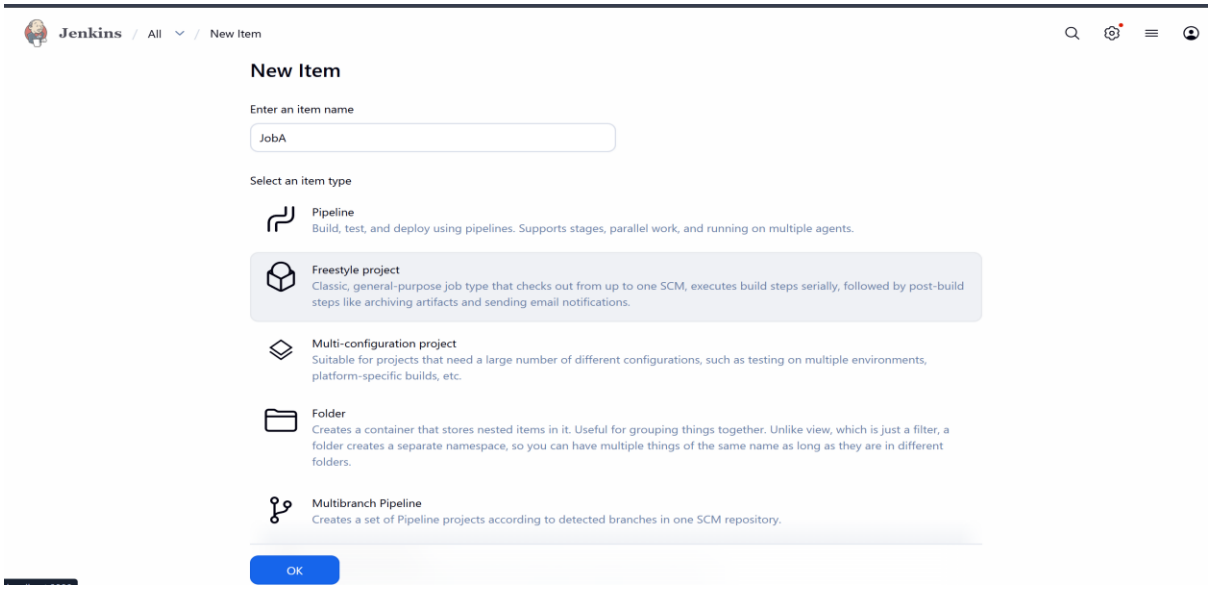
Learner connects Jenkins with GitHub.

3. Learning Goal

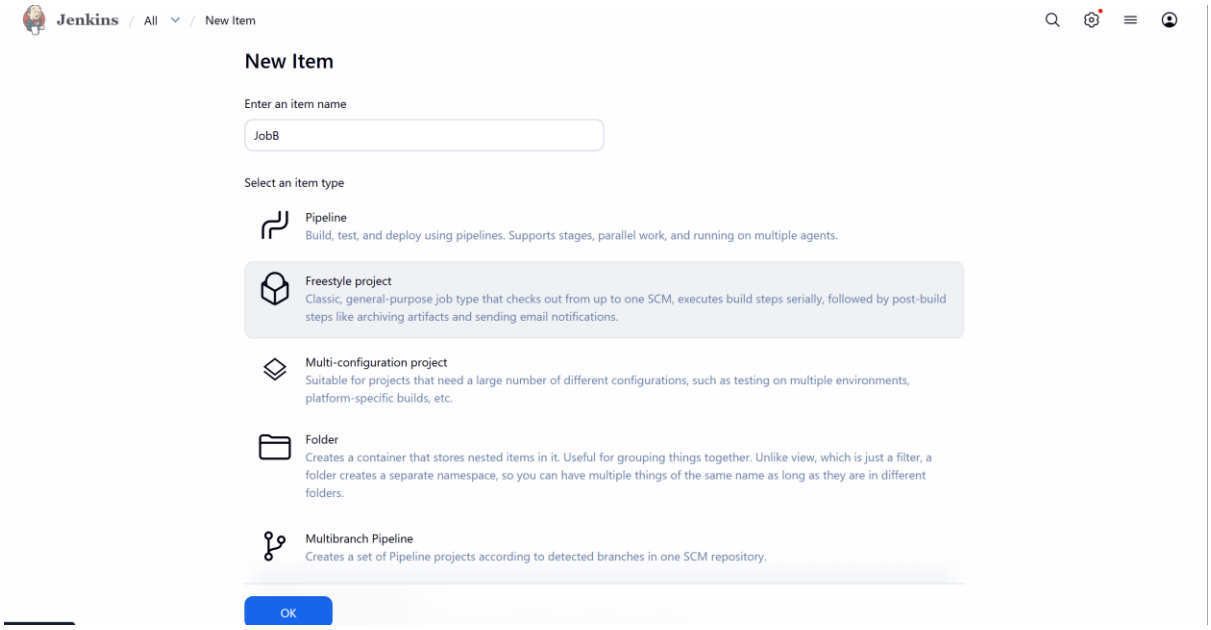
Stepwise Tasks

3.1.Create multiple Freestyle jobs:

- **Job-A (Build)**



- **Job-B (Test)**



- **Job-C (Deploy)**

3.2.Use:

- **Post-build action → Trigger other projects**

Jenkins / JobA / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Inspect build log for published build scans

Terminate a build if it's stuck

With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command ?

Command

See the list of available environment variables

```
echo Building the project
```

Advanced

+ Add build step

Save Apply

Jenkins / JobB / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Terminate a build if it's stuck

With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command ?

Command

See the list of available environment variables

```
echo Running tests
```

Advanced

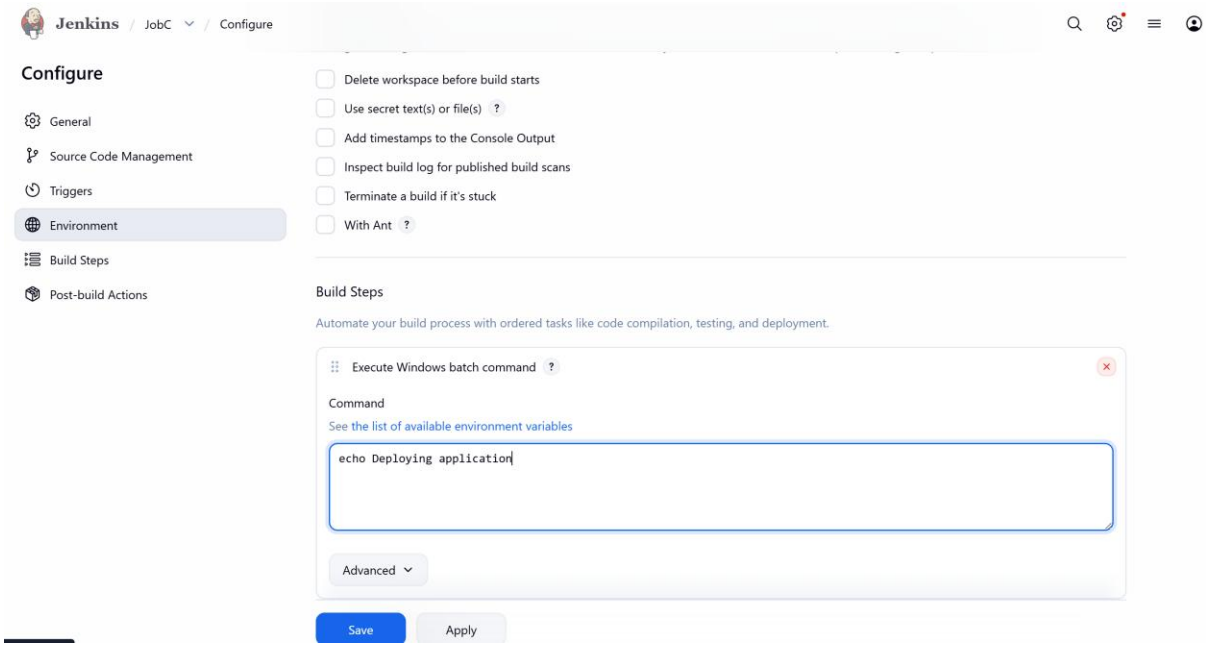
+ Add build step

Post-build Actions

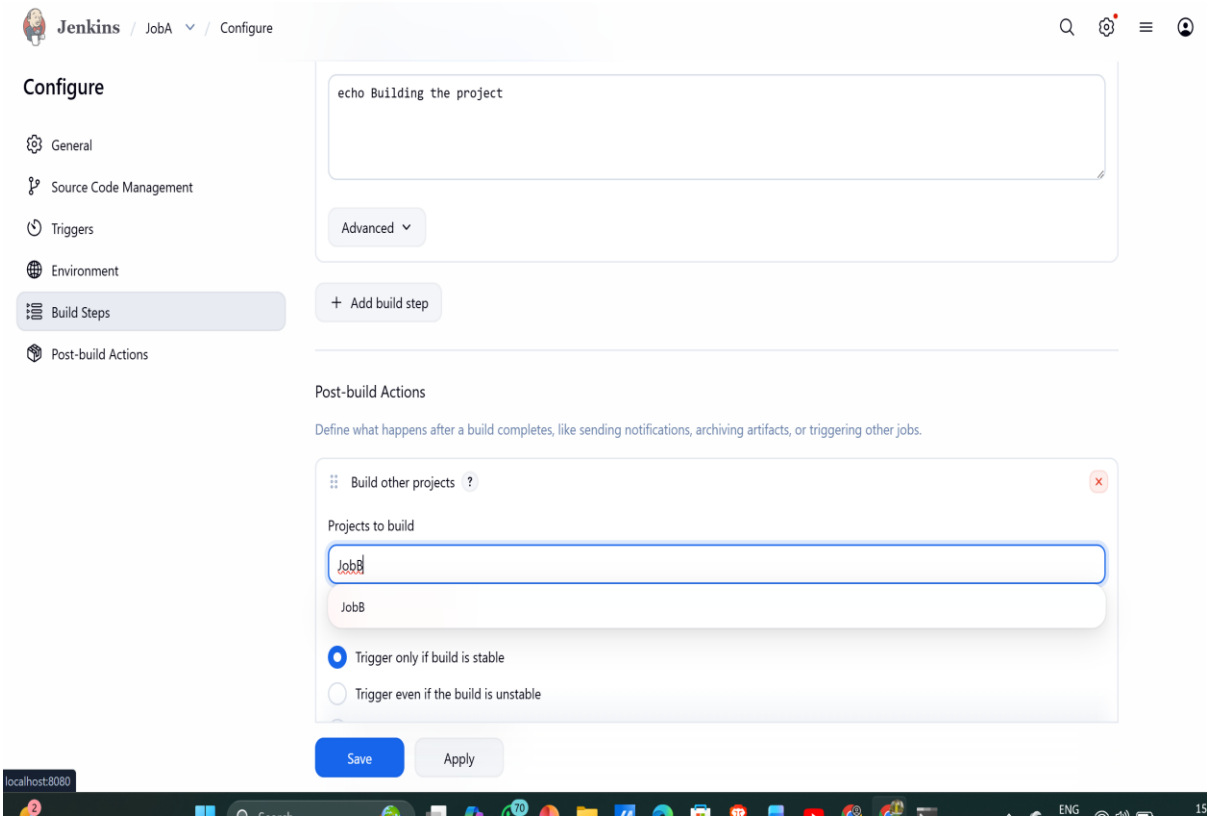
Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs

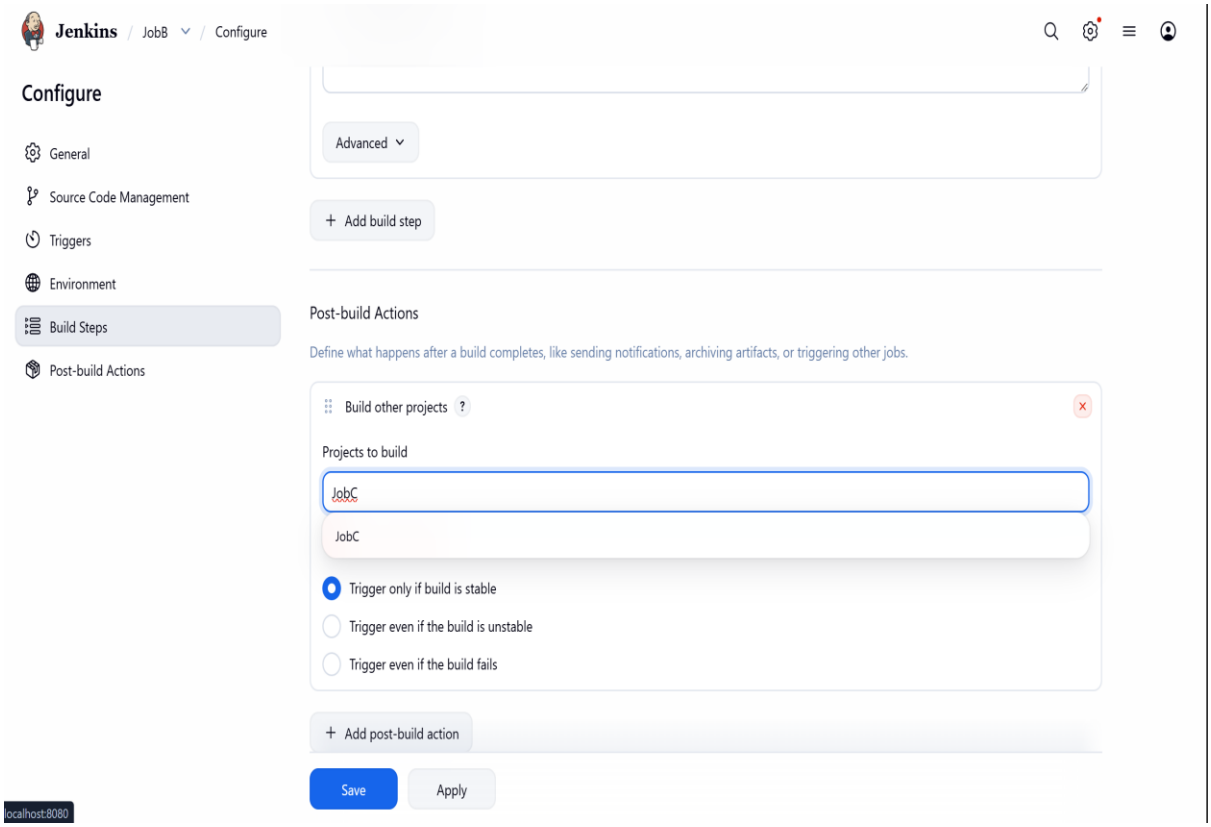
Save Apply

localhost:8080

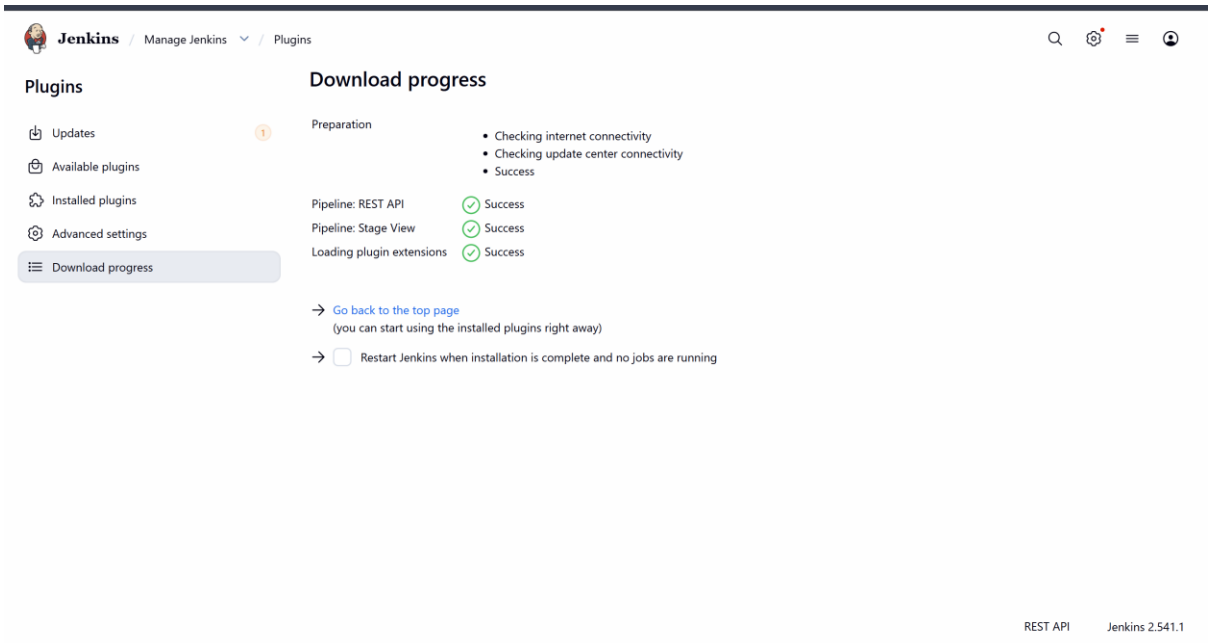


3.3. View job chaining visually

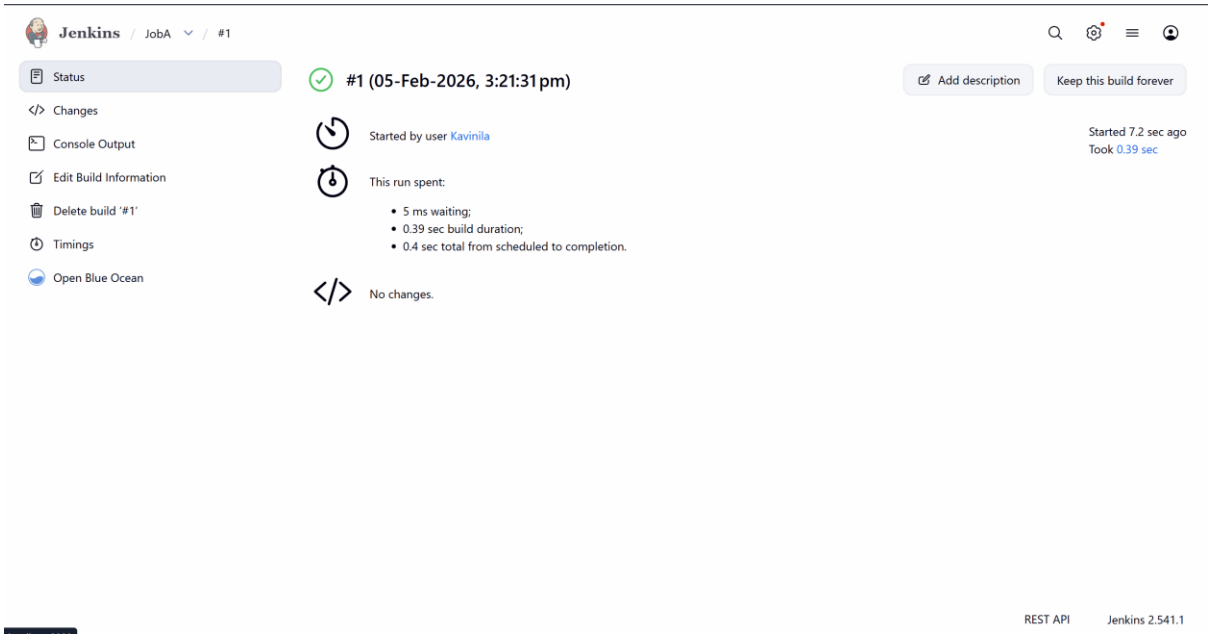




3.4.Install Pipeline Stage View



3.5Observe stage-wise execution



Outcome

Learner understands pipeline flow visually.

4. Learning Goal

Use Jenkins Pipeline & Jenkinsfile

Stepwise Tasks

4.1.What is Jenkins Pipeline?

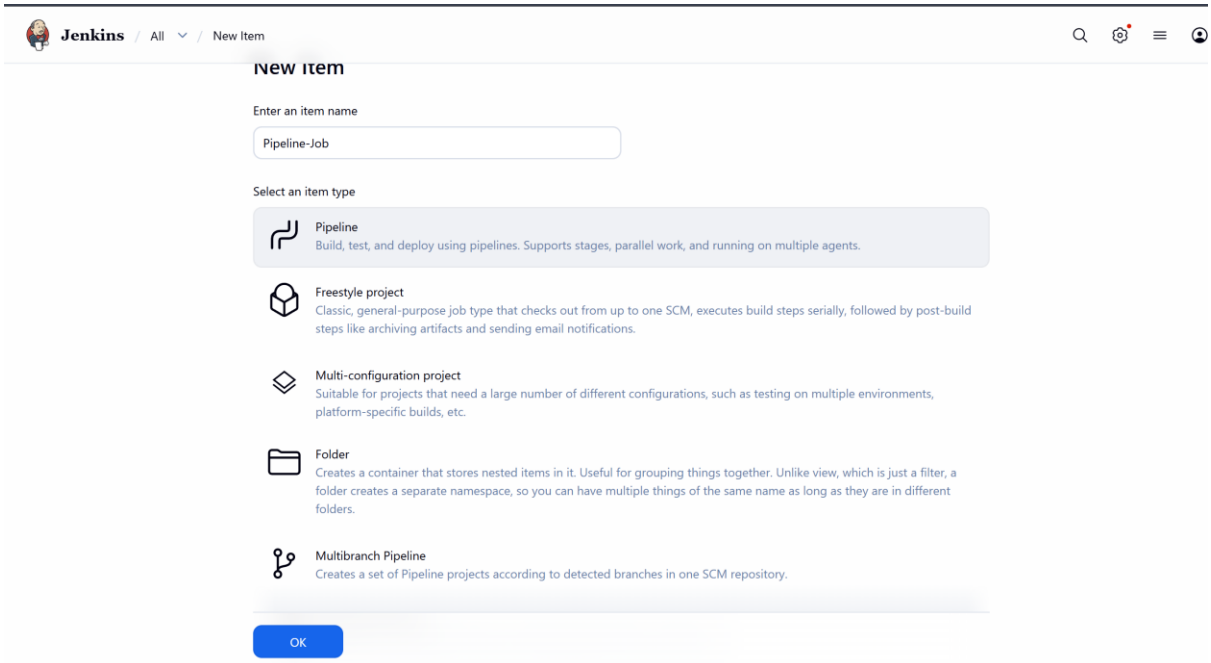
Pipeline = job written as code

Better than freestyle for CI/CD.

4.2.Scripted vs Declarative Pipeline

Type	Usage
Scripted	Advanced, Groovy-heavy
Declarative	Simple, recommended

4.3.Create Pipeline Job



4.4.Write pipeline in Jenkins UI

4.5.Add stages:

- **Build**
- **Test**
- **Deploy**

4.6.Introduce:

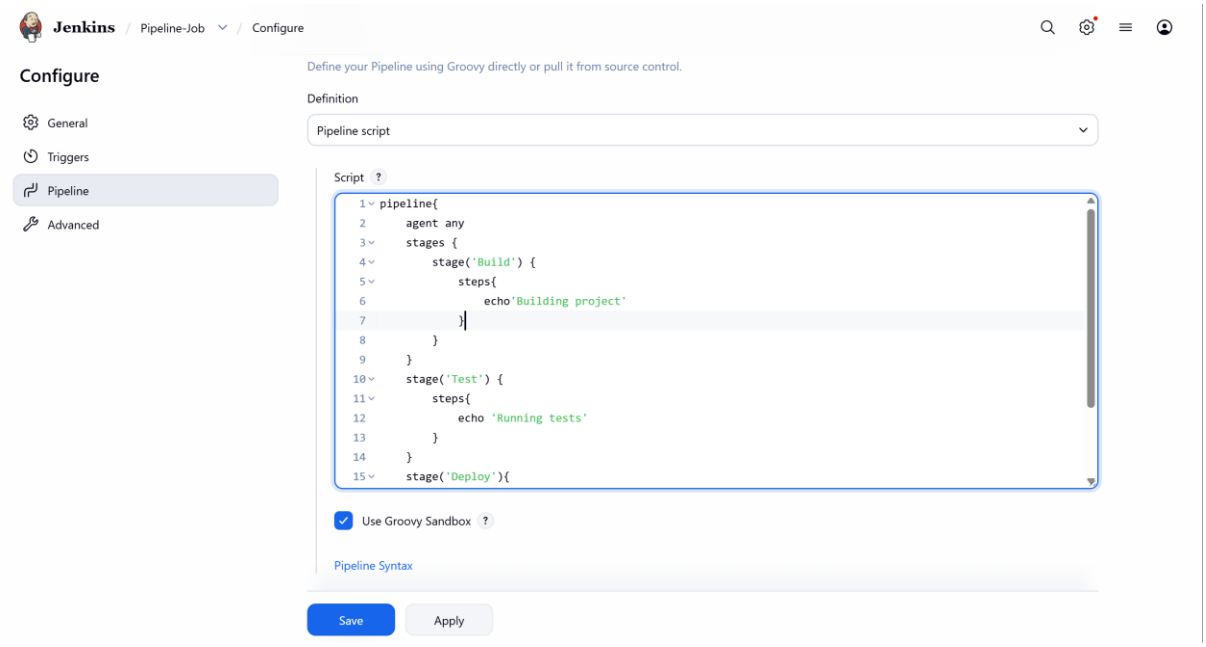
- `agent`
- `stages`
- `steps`

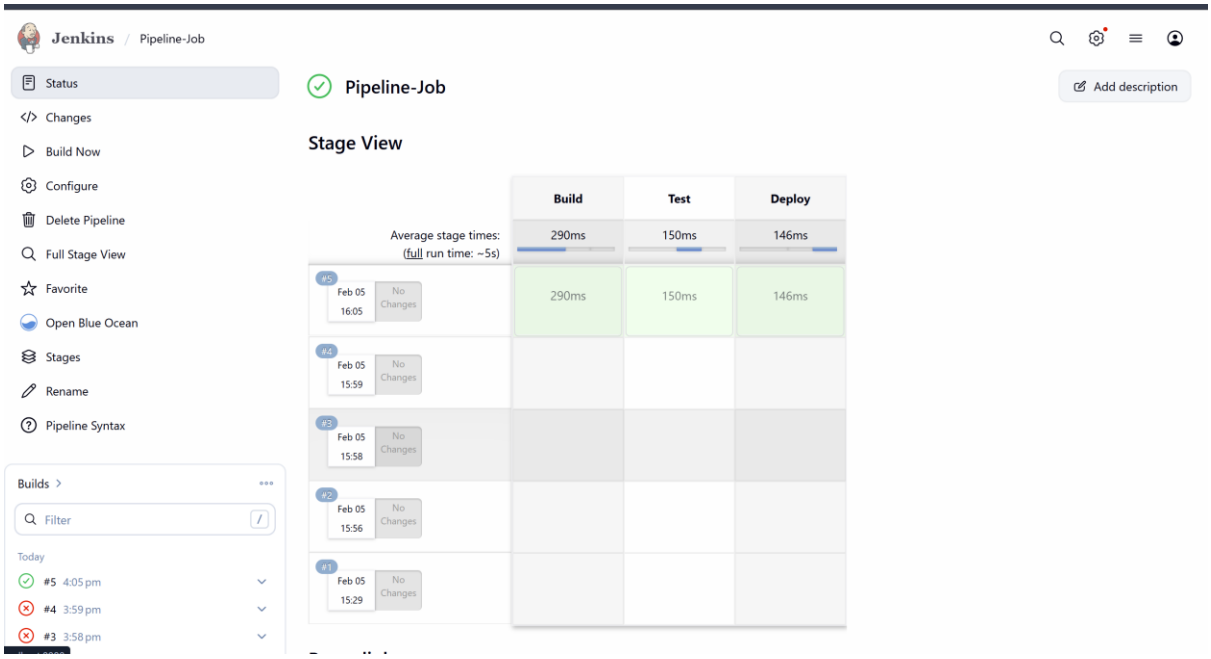
Sample Task

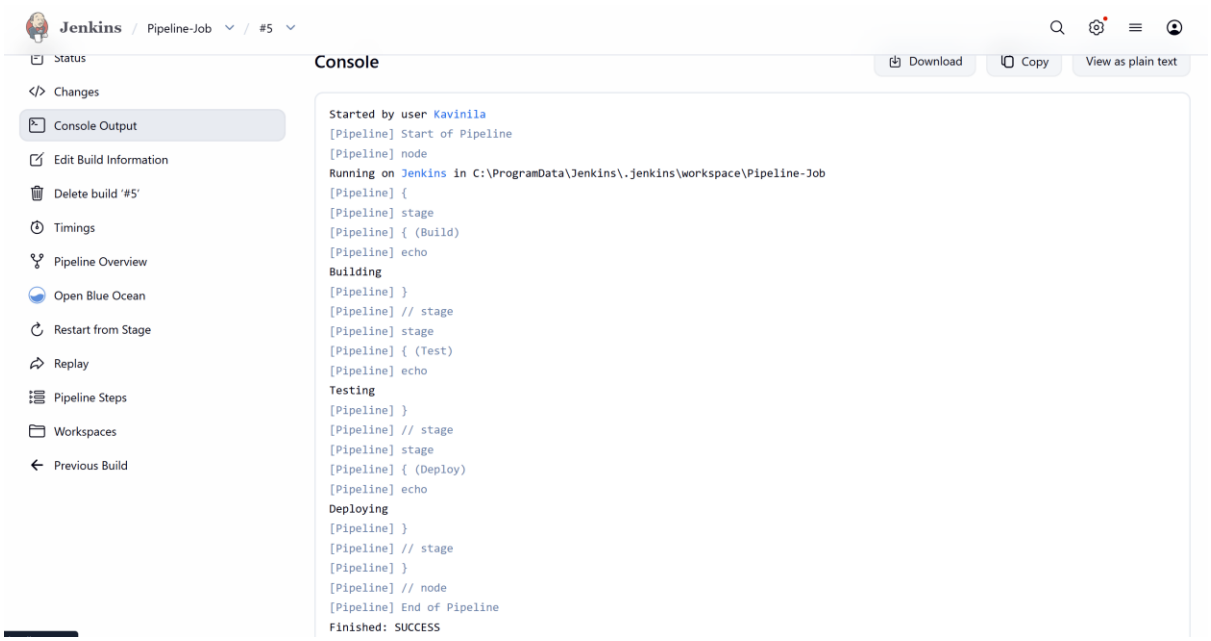
```
pipeline {
  agent any
  stages {
    stage('Build') { steps { echo 'Build' } }
    stage('Test')  { steps { echo 'Test' } }
    stage('Deploy'){ steps { echo 'Deploy' } }
  }
}
```

Outcome

Learner writes basic pipelines confidently.







5. Learning Goal

Pipeline as code from repository

Stepwise Tasks

5.1.Create `Jenkinsfile`

5.2.Commit Jenkinsfile to GitHub

5.3.Configure Pipeline job:

- **SCM → Git**

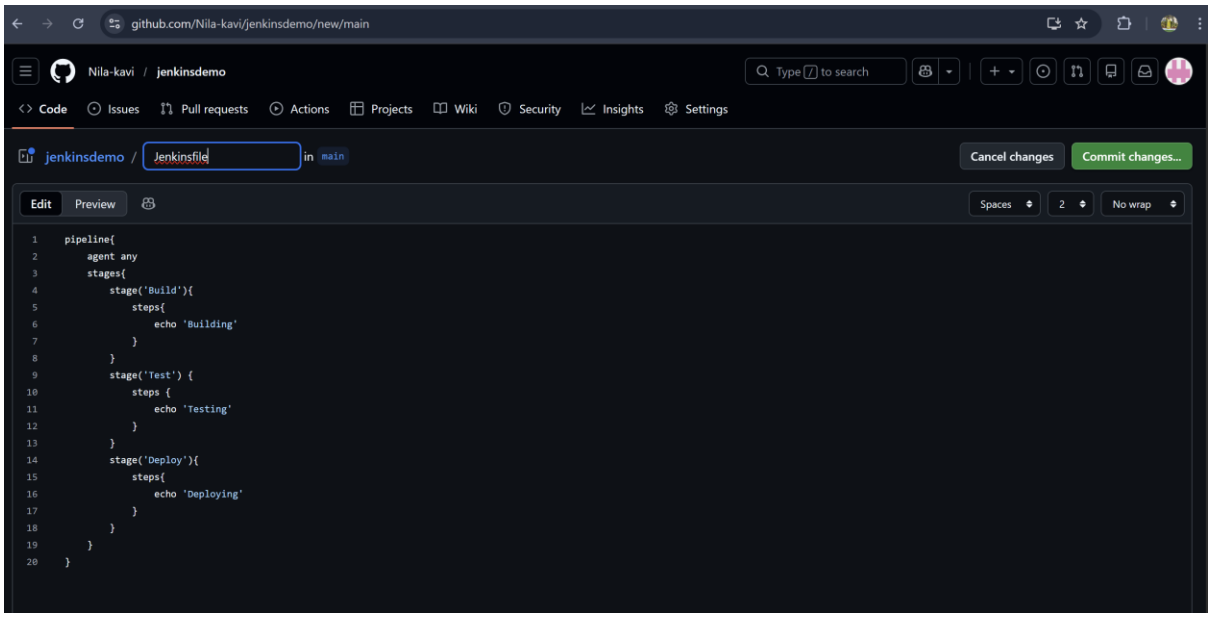
5.4.Trigger build automatically

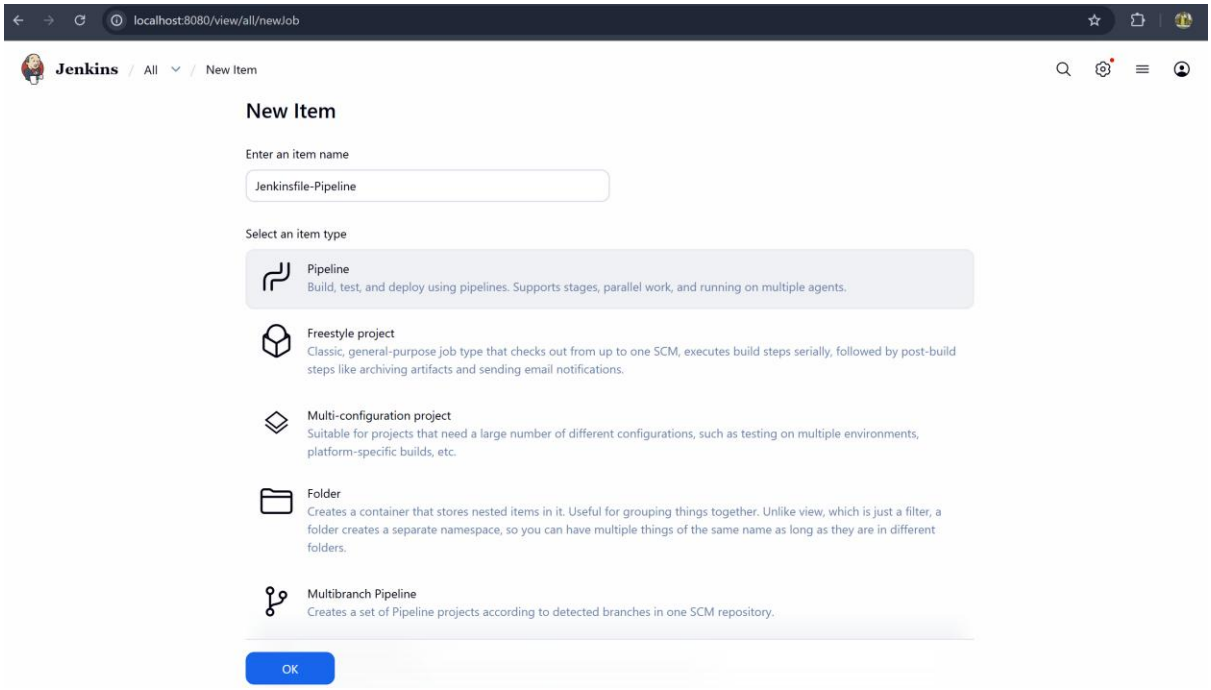
5.5.Modify Jenkinsfile and rebuild

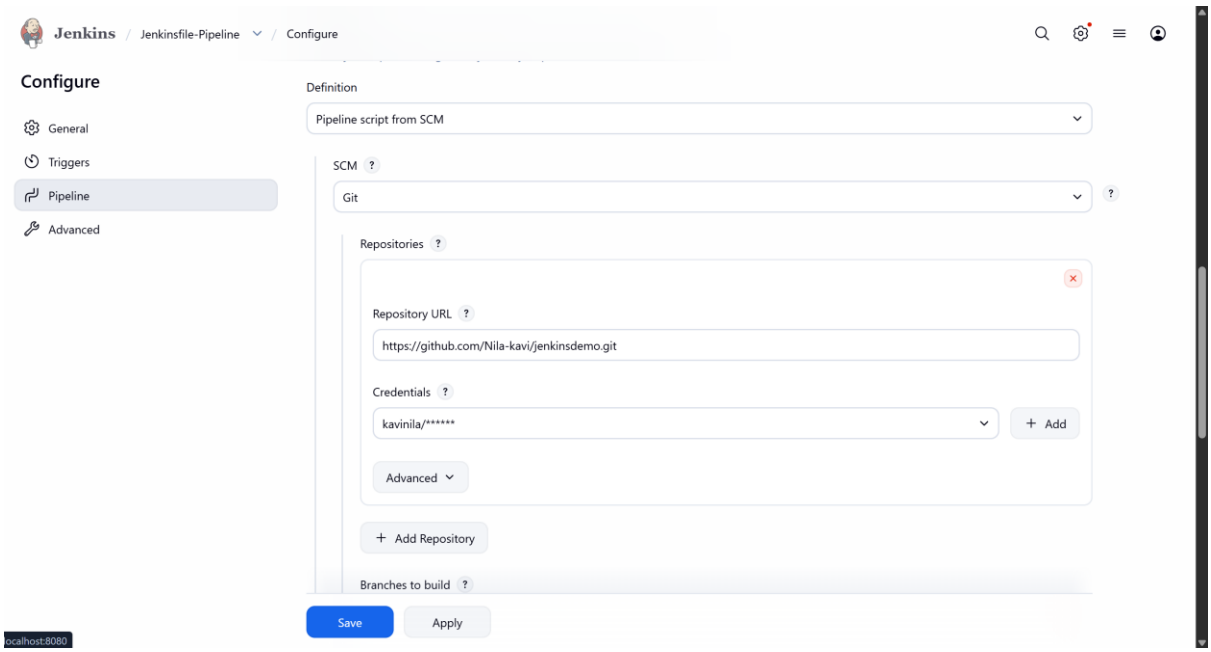
5.6.View Stage View

Outcome

Learner practices Infrastructure as Code concept.




localhost:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Jenkinsfile-Pipeline
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
OK


Jenkins / Jenkinsfile-Pipeline / Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/Nila-kavi/jenkinsdemo.git
Credentials
kavinila/******
Add
Advanced
Add Repository
Branches to build
Save
Apply
localhost:8080

Jenkins / Jenkinsfile-Pipeline / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

☑ Lightweight checkout ?

Pipeline Syntax

Save Apply

localhost:8080

Jenkins / Jenkinsfile-Pipeline

- Status
- Changes
- Build Now
- Configure
- Delete Pipeline
- Full Stage View
- Favorite
- Open Blue Ocean
- Stages
- Rename
- Pipeline Syntax

Builds

Today

#1 4:44 pm

Jenkinsfile-Pipeline

Add description

Stage View

	Declarative: Checkout SCM	Build	Test	Deploy
Average stage times: (full run time: ~17s)	3s	150ms	137ms	141ms
#1 Feb 05 16:44 No Changes	3s	150ms	137ms	141ms

Permalinks

Jenkins / Jenkinsfile-Pipeline / #1

```
Commit message: "Create Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```